

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO CAMPUS GUARULHOS

Caroliny Rocha Sampaio, GU3046532

Gabriel Vinícius Rocha Barboza, GU3042715

Jéssica Bueno Ramos - GU3046346

Extração de Microsserviço - API e Microsserviços

GUARULHOS - SP

2025

1. "Bounded Context" Escolhido

Bounded Context (ou *contexto delimitado*) é um limite bem definido dentro de um sistema onde um modelo de domínio é aplicado de forma coesa e isolada. Ele atua como uma solução específica para um conjunto de problemas de negócio, permitindo organizar melhor a lógica e a arquitetura da aplicação.

Ao aplicar esse conceito, o sistema é dividido em módulos independentes, cada um com sua própria linguagem ubíqua, regras de negócio, base de dados e responsabilidades, o que facilita a manutenção e a evolução da aplicação.

Neste projeto, optamos por extrair e implementar o Bounded Context de Autenticação e Autorização, com base nos seguintes fatores:

- Alta taxa de mudança: funcionalidades como login, cadastro, recuperação de senha e segurança evoluem com frequência, exigindo alterações constantes (como integração com OAuth, autenticação multifator ou expiração de tokens).
- Reutilização em outros contextos: o serviço de autenticação pode ser aproveitado por outros sistemas além do catálogo de filmes, promovendo padronização e economia de esforço.
- Criticidade: segurança e controle de acesso são componentes essenciais para a confiabilidade do sistema, tornando sua separação estratégica.
- Escalabilidade isolada: permite escalar o serviço de autenticação de forma independente do restante da API, o que é útil em cenários com alto volume de logins ou validações de token.

Essa escolha contribui para uma arquitetura mais modular, segura e preparada para crescimento futuro.

2. Context Map

Para garantir uma arquitetura coesa e com separação clara de responsabilidades, foi elaborado o seguinte Context Map da aplicação de catálogo de filmes, considerando a autenticação como um microsserviço separado. Cada contexto representa um subconjunto de funcionalidades do domínio, com fronteiras bem definidas e comunicação estruturada.

Auth Context

O Auth Context é responsável pela autenticação de usuários, por meio de login e emissão de token JWT. Esse contexto foi isolado como um microsserviço independente, acessado via chamadas REST. Ele se comunica com os demais contextos utilizando o token JWT em requisições autenticadas. Suas principais responsabilidades são o login de usuários e a geração/validação de tokens. A integração com os demais contextos ocorre via REST, especialmente com Movie Catalog, Watchlist e Review.

User Context

O User Context gerencia os dados dos usuários cadastrados na plataforma, incluindo nome, e-mail, username, senha e o papel de acesso (USER ou ADMIN). Esse contexto compartilha dados comuns com outros módulos, como o identificador do usuário e seu perfil de acesso. Suas responsabilidades incluem o cadastro de usuários, a atribuição de papéis (roles) e o controle de permissões. A integração ocorre por meio de um kernel compartilhado (Shared Kernel), garantindo que os dados de usuário sejam consistentes em todos os contextos da aplicação, como nos módulos de filmes, avaliações e watchlist.

Movie Catalog Context

O Movie Catalog Context é o núcleo da aplicação, responsável pelo gerenciamento completo do catálogo de filmes. Ele contempla funcionalidades de cadastro, edição, exclusão e listagem de filmes, com suporte a filtros por título, gênero e ano. Também expõe dados do catálogo de forma pública para consumo pelos demais contextos. Este contexto é consumido diretamente por Watchlist, Review e Recommendation

Contexts, que utilizam seus dados como referência. A integração com o Auth Context é feita via REST, garantindo que apenas usuários autenticados acessem as funcionalidades protegidas.

Watchlist Context

O Watchlist Context permite que usuários criem uma lista de filmes que desejam assistir futuramente. Também possibilita que os usuários marquem filmes como "assistidos". A principal responsabilidade deste contexto é manter o histórico de interesse e consumo dos usuários, fornecendo informação crucial para outros módulos, como o Review Context. Ele se integra ao Movie Catalog Context via REST, para validar a existência dos filmes adicionados. Além disso, compartilha com o Review Context os dados sobre quais filmes foram efetivamente assistidos, permitindo ou bloqueando avaliações conforme necessário.

Review Context

O Review Context gerencia as avaliações de filmes realizadas pelos usuários. Apenas filmes previamente marcados como "assistidos" no Watchlist Context podem ser avaliados. Este contexto suporta avaliações com múltiplos critérios (roteiro, direção, fotografia) e nota geral. Também permite edição e exclusão de avaliações, além de funcionalidades sociais como curtidas e denúncias. Avaliações com excesso de denúncias podem ser ocultadas automaticamente. Ele se integra com o Watchlist Context para validar se o filme foi assistido, com o Movie Catalog Context para verificar a existência do filme, e com o Recommendation Context, para envio de eventos de avaliação.

Recommendation Context

O Recommendation Context é responsável por fornecer sugestões de filmes aos usuários com base em seu histórico de avaliações e gêneros favoritos. Suas responsabilidades incluem a geração de recomendações personalizadas, cálculo de rankings dos filmes mais bem avaliados e fornecimento de estatísticas de engajamento. Esse contexto funciona principalmente de forma assíncrona, processando eventos disparados pelo Review Context (como novas avaliações) e pelo Watchlist Context (como filmes assistidos). Além disso, realiza leituras

agregadas dos dados do Movie Catalog e User Context para enriquecer suas análises.

Estilo de Comunicação entre Contextos

A comunicação entre os contextos varia conforme a necessidade. Para ações como autenticação, cadastro de usuários, consulta de filmes e alteração de listas, a comunicação ocorre de forma síncrona, por meio de chamadas REST entre os contextos Auth, User, Movie Catalog e Watchlist. O Shared Kernel é utilizado entre o User Context e os demais módulos para compartilhar dados essenciais como identificadores e perfis de usuário. Já a comunicação entre os contextos Review, Recommendation e Watchlist também envolve integração assíncrona baseada em eventos. Por exemplo, quando um usuário avalia um filme ou o marca como assistido, eventos são disparados e consumidos por outros contextos. A estratégia de ownership com referência é utilizada nos contextos Watchlist e Review, onde os filmes são referenciados a partir do Movie Catalog Context sem duplicação de dados.

Observação: Context Map disponível em versão .txt no repositório e na pasta compactada.

3. Linguagem Ubíqua e Modelagem Tática

Para garantir uma comunicação clara e consistente, foi adotada a linguagem ubíqua no desenvolvimento do módulo de autenticação do sistema. Todos os termos utilizados no código, nos testes, nos commits e na documentação seguem o vocabulário do domínio de autenticação, facilitando o alinhamento conceitual e evitando ambiguidade.

No processo de autenticação, utilizamos o termo usuário (User) para representar a pessoa que acessa o sistema. Cada usuário possui informações como nome, e-mail, nome de usuário (username) e senha (password). Durante o login, o usuário fornece suas credenciais de acesso, compostas por nome de usuário e senha, que são encapsuladas em um objeto AuthenticationDTO – também chamado de solicitação de autenticação. Esse DTO é enviado ao endpoint /api/auth controlado pela classe AuthenticationController.

O método authenticate recebe essas credenciais e, com o auxílio da classe AuthenticationService, executa a validação de autenticação. Se os dados estiverem corretos, o sistema retorna um token de acesso – uma string JWT – que representa a autorização temporária daquele usuário para interagir com recursos protegidos do sistema. Caso contrário, o acesso é negado.

Optou-se por manter os nomes em inglês, como username, password, authenticate e AuthenticationService, respeitando o vocabulário técnico amplamente compreendido pelo time e evitando traduções forçadas que poderiam prejudicar a clareza. Expressões genéricas ou técnicas excessivas, como usr_fk ou dt_login, foram evitadas em favor de nomes mais próximos do domínio, como username e password.

Além disso, termos foram padronizados em todas as camadas da aplicação: entidades, DTOs, serviços, testes, documentação e mensagens de commit utilizam os mesmos nomes e conceitos. Por exemplo, os testes seguem a convenção shouldReturnAccessToken_WhenCredentialsAreValid() e as mensagens de commit refletem claramente o domínio, como feat(auth): permitir autenticação com username e password.

Essa padronização fortalece a comunicação entre desenvolvedores, analistas e stakeholders, além de facilitar a manutenção e evolução do sistema. Ao empregar a linguagem ubíqua com disciplina, garantimos que todos os envolvidos compartilhem o mesmo modelo mental e falem a mesma língua, tanto no código quanto fora dele.

4. Contrato do Microserviço de Autenticação e Autorização

4.1 Endpoints REST

O microserviço expõe dois endpoints REST principais, responsáveis por registrar novos usuários e autenticar logins. Todos os endpoints seguem o padrão `/api`.

Método	Endpoint	Descrição
POST	<code>/api/auth</code>	Autentica um usuário e retorna um token JWT
POST	<code>/api/users/register</code>	Registra um novo usuário com papel padrão ROLE_USER

POST `/api/auth`

- Autentica um usuário existente.
- Corpo da Requisição (AuthenticationDTO):

```
{  
  "username": "testuser",  
  "password": "UserPass123!"  
}
```

Resposta:

- 200 OK: Retorna um token JWT como String.

- 401 Unauthorized: Credenciais inválidas.

POST /api/users/register

- Registra um novo usuário no sistema. Atribui o papel padrão ROLE_USER.
- Corpo da Requisição (UserRequestDTO)

```
{  
  
  "name": "Test User",  
  
  "email": "testuser@example.com",  
  
  "username": "testuser",  
  
  "password": "UserPass123!"  
}
```

Resposta:

- 201 Created: Retorna os dados do usuário criado.
- 400 Bad Request: Dados inválidos ou e-mail já cadastrado.

4.2 DTOs Utilizados

- AuthenticationDTO: username e senha;
- UserRequestDTO: name, e-mail, username, password;
- UserResponseDTO: id, name, email, username, roles

5. Adaptação do Monólito

No monólito original do Catálogo de Filmes, foram realizadas algumas adaptações importantes para integrar o novo microsserviço de autenticação. Primeiramente, o código de autenticação local foi removido, delegando toda a responsabilidade de login, validação de credenciais e geração de tokens ao microsserviço externo. Em seguida, foi criado um cliente REST — utilizando RestTemplate ou WebClient — para permitir a comunicação com o Auth Service. Esse cliente é utilizado tanto para validar tokens em requisições protegidas quanto para obter os dados do usuário autenticado. Além disso, foi implementado um filtro (ou interceptor JWT) no monólito, responsável por interceptar as requisições e consultar o endpoint `/auth/validate` do microsserviço, garantindo que apenas usuários devidamente autenticados possam acessar os recursos protegidos da aplicação.

6. Repositórios de Código

- **Microserviço Auth:**

<https://github.com/Carolinyr9/auth-service/tree/bugs-jessy>

- **Monólito ajustado :**

<https://github.com/Carolinyr9/my-movinhos/tree/bugs-jessy>

Obs.: acessar branch bugs-jessy